

Properties of a Good Algorithm:

1. **Correctness:** The algorithm must produce the correct output for all valid inputs. It should solve the problem accurately according to its specification.
2. **Finiteness:** The algorithm must terminate after a finite number of steps. It should not go into an infinite loop.
3. **Definiteness:** Each step of the algorithm must be precisely and unambiguously defined. There should be no room for subjective interpretation.
4. **Input:** An algorithm should have zero or more well-defined inputs, which are the quantities that are given to the algorithm initially before it begins.
5. **Output:** An algorithm should have one or more well-defined outputs, which are the quantities that have a specified relationship to the inputs. These are the result of the algorithm.
6. **Efficiency:** The algorithm should be efficient in terms of the computational resources it uses, primarily time (how fast it runs) and space (how much memory it uses).
7. **Generality:** The algorithm should be applicable to a range of inputs, not just a single specific case. (Sometimes also called Robustness or Portability).
8. **Clarity and Readability:** The algorithm should be easy to understand and follow. This is important for verification, debugging, and modification.

Influence on Efficiency and Effectiveness:

These properties significantly influence both the efficiency and effectiveness of an algorithm:

1. **Correctness:** This is fundamental to **effectiveness**. An algorithm that doesn't produce the correct output, regardless of how fast it runs, is not effective in solving the problem. Correctness ensures the algorithm actually does what it's supposed to do.
2. **Finiteness:** This is also crucial for **effectiveness**. An algorithm that never terminates cannot provide a solution and is therefore ineffective. Finiteness guarantees that a solution (or a determination that no solution exists) will be reached within a predictable timeframe, even if that timeframe is long.
3. **Definiteness:** Precision in each step contributes to both **effectiveness** and helps in analyzing **efficiency**. Ambiguous steps can lead to incorrect execution, making the algorithm ineffective. Clear steps make it easier to analyze the computational cost (time and space) accurately.
4. **Input and Output:** Clearly defined inputs and outputs are essential for **effectiveness**. Without knowing exactly what the algorithm receives and what it is expected to produce, it's impossible to verify if it works correctly. They also help in determining the scope and scale of the problem, which is relevant for analyzing **efficiency** (e.g., how runtime scales with input size).
5. **Efficiency (Time and Space Complexity):** This property directly dictates the **efficiency** of the algorithm.
 - **Time Efficiency:** A more time-efficient algorithm solves the problem faster, which is critical for applications with performance requirements or large datasets.
 - **Space Efficiency:** A more space-efficient algorithm uses less memory, which is important when memory is limited or when dealing with large amounts of data that might not fit into available memory otherwise.
6. **Generality:** While not strictly about speed or correctness for a *single* instance, generality contributes to the overall **effectiveness** and utility of the algorithm. A general algorithm can solve a class of problems, making it more valuable and reusable. It influences **efficiency** in that a general algorithm might be optimized for a wider range of inputs, potentially leading to better average performance across different scenarios within its scope.
7. **Clarity and Readability:** While not directly a measure of runtime or memory usage, clarity significantly impacts the ability to analyze and improve both **efficiency** and **effectiveness**. A clear algorithm is easier to:
 - Verify for **correctness**.
 - Identify potential bottlenecks or areas for optimization to improve **efficiency**.
 - Debug when it doesn't work correctly.
 - Modify or adapt for related problems. Poor clarity can hide errors or inefficiencies, making the algorithm less effective and potentially less efficient in practice due to development and maintenance overhead.